



DDBMS

Design Issues



OUTLINE

- ✓ Transparencies in DDBMS
- ✓ Distributed Transaction Management
- ✓ Concurrency Control in DDBMS
- ✓ Distributed Deadlock Management



Transparencies in DDBMS

□ Transparency

- It refers to the separation of the high-level semantics of a system from **lower-level implementation issues**. In a distributed system, it hides the implementation details from users of the system.
- The user believes that he/she is working with a centralized database system and that all the complexities of a distributed database system are either hidden or transparent to the user.
- Four main categories of transparencies:
 - Distribution transparency
 - Transaction transparency
 - Performance transparency
 - DBMS transparency



Transparencies in DDBMS

❑ Distribution transparency

- Distribution transparency refers to the degree or extent to which details of fragmentation, replication and location are hidden from users.

- Distribution transparency can be classified into:
 - Fragmentation transparency
 - Location transparency
 - Replication transparency
 - Local mapping transparency
 - Naming transparency



Transparencies in DDBMS

❑ Fragmentation transparency

- It hides the fact from users that the data are fragmented. Users are not required to know how a relation has been fragmented. To retrieve data, the user needs not to specify the particular fragment names.

❑ Location transparency

- Users are not required to know the physical location of the data. To retrieve data from a distributed database with location transparency, the user has to specify the database fragment names but need not to specify where the fragments are located in the system.



Transparencies in DDBMS

❑ Replication transparency

- The user is unaware of the fact that the fragments of relations are replicated and stored in different sites of the system.

❑ Local mapping transparency

- It refers to the fact that users are aware of both the fragment names and the location of fragments, taking into account that any replication of the fragments may exist. In this case, the user has to mention both the fragment names and the location for data access.



Transparencies in DDBMS

❑ Naming transparency

- In a distributed database system, each database object such as- relations, fragments, replicas etc, must have a unique name. Therefore, the DDBMS must ensure that no two sites create a database object with the same name. **One solution to this problem is to create a central name server, which has the responsibility to ensure uniqueness of all names in the system.** Naming transparency means that the users are not aware of the actual name of the database object in the system. In this case, the user will specify the alias names of the database objects for data accessing.



Transparencies in DDBMS

❑ Transaction Transparency

- This ensures that all distributed transactions maintain database integrity and consistency.
- Each transaction is **divided into sub-transactions**.
- Each sub transaction can update data stored at different sites.
- **Transaction transparency ensures that the distributed transaction will be successfully completed only if all sub-transactions are completed successfully.**



Transparencies in DDBMS

❑ Performance Transparency

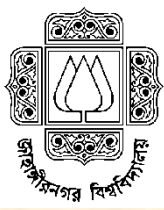
- Performance transparency requires a DDBMS to perform as if it were a centralized DBMS.
- The system shouldn't suffer from any performance degradation due to the distributed architecture.



Transparencies in DDBMS

❑ DBMS Transparency

- This is also known as heterogeneity transparency.
- DBMS transparency hides the knowledge that the local DBMSs may be different.
- Only applicable to heterogeneous DDBMSs.
- This allows the integration of several local DBMSs under a common global schema.
- It is one of the most difficult transparencies to provide as a generalization.



Distributed Transaction Management

□ Transaction

- A transaction consists of a set of operations that perform a single logical unit of work in a database system and moves the database from one consistent state to another. It may be an entire program or a part of a program.

□ ACID properties of transactions

- **A - Atomicity**
- **C - Consistency**
- **I - Isolation**
- **D - Durability**



Distributed Transaction Management

❑ ACID properties of transactions

- **Atomicity.** All changes to data are performed as if they are a single operation. That is, all the changes are performed, or none of them are.
- **Consistency.** A transaction must transform the database from one consistent state to another consistent state. It can ensure consistency by enforcing all the constraints specified in the database schema.



Distributed Transaction Management

❑ ACID properties of transactions

- **Isolation.** Transaction can execute independently. They are not dependent on each other. In other words, transactions are isolated from one another and the partial effect of incomplete transactions should not be visible to other transactions.
- **Durability.** It refers to the fact that the effects of a successfully completed (committed) transaction are permanently recorded in the database and will not be affected by a failure. This ensures that once a transaction commits, the changes are durable and cannot be erased from the database.



Distributed Transaction Management

❑ Objectives of Distributed Transaction Management

- **CPU and main memory utilization should be improved.** Most of the typical database applications spend much of their time waiting for I/O operations rather than on computations. To improve CPU and main memory utilization, a transaction manager should adopt specialized techniques.
- **Response time should be minimized.** To improve the performance of transaction executions, the response time of each individual transaction must be considered and should be minimized.



Distributed Transaction Management

❑ Objectives of Distributed Transaction Management

- **Availability should be maximized.** Although the availability in a distributed system is better than that in a centralized system, it must be maximized for transaction recovery and concurrency control.
- **Communication cost should be minimized.** In distributed systems, an additional communication cost is incurred, because a number of message transfers are required between sites to control the execution of a global transaction. Preventative measures should be adopted by the transaction manager to minimize the communication cost.



Distributed Transaction Management

□ A Model for Transaction Management in DDBMS

- Access to the various data items in a distributed system is usually accomplished through transactions which must preserve the ACID properties. There are two types of transaction that we need to consider.
 - The local transactions are those that access and update data in only local database.
 - The global transactions are those that access and update data in several local databases.



Distributed Transaction Management

□ A Model for Transaction Management in DDBMS

- Ensuring the ACID properties of local transactions can be done easily. However, for global transactions, this task is much more complicated, since several sites are participating in execution. A model for transaction management at each site of a distributed system is shown below.

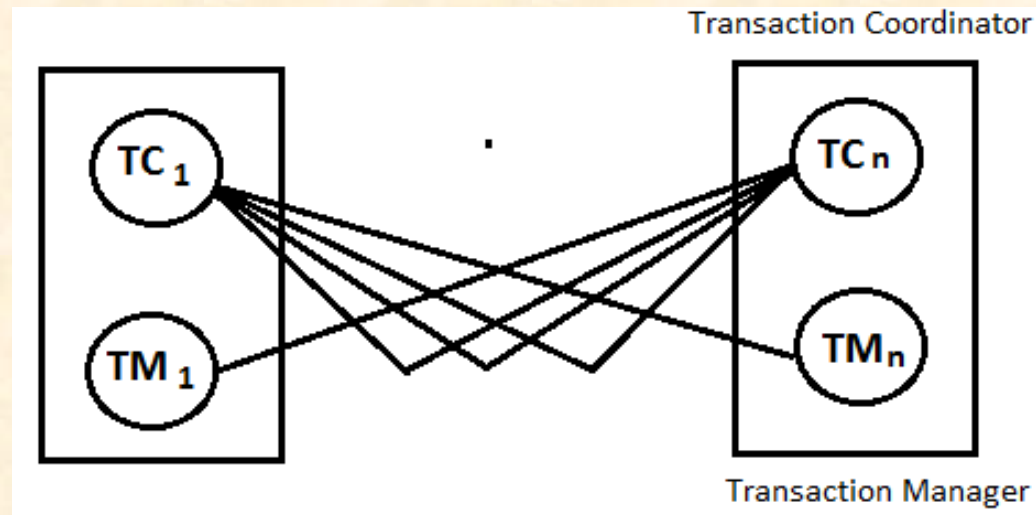


Fig. A Model for Transaction Management at each site in a DDBMS



Distributed Transaction Management

❑ A Model for Transaction Management in DDBMS

- It consists of two sub-modules:
 - Transaction Manager (TM) and
 - Transaction Coordinator (TC)

❑ Responsibility of Transaction Manager

- It manages the execution of those transactions that access data stored in a local site. Each such transaction may be either a local transaction or a part of a global transaction.
- It ensures the ACID properties of those transactions that execute at that site.
- It maintains a log for recovery purpose.
- It participates in concurrency control scheme to coordinate the concurrent execution of transactions executing at that site.



Distributed Transaction Management

❑ Responsibility of Transaction Coordinator

- It coordinates the execution of the various transactions both local and global initiated at that site.
- It breaks the transaction into a number of sub-transactions and distributes these sub-transactions to the appropriate sites for execution.
- It coordinates the termination of the transactions which may result in the transaction being committed at all sites or aborted at all sites.



Concurrency Control in DDBMS

□ Concurrency Control

- It is the activity of coordinating concurrent access to data items in a distributed database environment.
- In a distributed database system, users may access data stored at many different sites, and a concurrency control mechanism at one site may not be aware of transactions at other sites instantaneously. Again the replication of data items add extra complexity to the concurrency control mechanism.



Concurrency Control in DDBMS

❑ Objectives of Distributed Concurrency Control

A good concurrency control mechanism for a DDBMS has the following objectives:

- It must be resilient to site and communication failures.
- It should permit parallel execution of transactions to achieve maximum concurrency.
- Its computational methods and storage mechanisms should be modest to minimize overhead.
- It should perform satisfactorily in a network environment taking into consideration that it involves significant communication delay.
- It must impose few constraints on the structure of atomic actions of transactions.
- It must preserve the consistency of data items.



Concurrency Control in DDBMS

❑ Concurrency Control Anomalies

Different anomalies can arise due to concurrent access of data:

- **Lost update anomaly** – This occurs when a successful completed update operation made by one transaction is overridden by another transaction.
- **Uncommitted dependency** – This problem occurs when one transaction allows other transactions to read its data before it has committed and then decides to abort.
- **Inconsistent analysis anomaly** – The problem occurs when a transaction reads several values from the database but a second transaction updates some of them during the execution of the first.



Concurrency Control in DDBMS

❑ Concurrency Control Anomalies

Different anomalies can arise due to concurrent access of data:

- **Phantom read anomaly** – This anomaly occurs when a transaction performs some operation on the database based on a selection predicate, another transaction inserts new tuples satisfying that predicate into the same database. This is known as phantom read.
- **Multiple-copy consistency problem** – This occurs when data items are replicated and stored at different sites. To maintain the consistency, when a replicated data item is updated at one site, all other copies must be updated. Otherwise, the database becomes inconsistent.



Concurrency Control in DDBMS

❑ Locking-based Concurrency Control Protocols

- A lock is nothing but a mechanism that tells the DBMS whether a particular data item is being used by any transaction for read/write purpose. Since there are two types of operations, i.e. read and write, whose basic nature are different, the locks for read and write operation may behave differently.
- In this locking-based approach, before accessing any data item, a transaction must acquire a lock on that data item. When a transaction acquires a lock on a data item, the lock prevents other transactions from modifying that data item.



Concurrency Control in DDBMS

❑ Locking-based Concurrency Control Protocols

- There are two types of locks:
 - read lock also known as **shared lock** and
 - write lock also known as **exclusive lock**.
- **Read lock:** If a transaction obtains a read lock on a data item, it can only read but cannot update that data item. In this case, other transactions are allowed to read that data item. Because read-read operation is **non-conflicting**. Therefore, several transactions can acquire read lock on same data item simultaneously.
- **Write lock:** If a transaction obtains a write lock on a data item, it can read as well as update that data item. In this case, no other transactions are allowed to read or update that data item. Because read-write and write-write operations are **conflicting**.



Concurrency Control in DDBMS

❑ Locking-based Concurrency Control Protocols

Basic Operation: In a distributed database system, the **lock manager** is responsible for managing locks for different transactions that are running on that system. When any transaction requires a lock on data items, the **transaction manager** passes this request to the lock manager. The lock manager checks whether that data item is currently locked by another transaction or not. If the data item is locked by another transaction and the locking mode is incompatible with the lock requested by the current transaction, the lock manager does not allow the current transaction to obtain the lock. Hence, the transaction is delayed until the existing lock is released. Otherwise, the lock manager permits the current transaction to obtain the desired lock and the information is passed to the transaction manager.



Concurrency Control in DDBMS

❑ Locking-based Concurrency Control Protocols

Up-gradation of lock

Some systems initially allow the current transaction to acquire a read lock on a data item, if it is compatible with the existing lock and later the lock is converted into a write lock. This is called up-gradation of lock.

Down-gradation of lock

To allow maximum concurrency, some systems permit the current transaction to acquire a write lock on a data item, and later the lock is converted into a read lock. This is called down-gradation of lock.



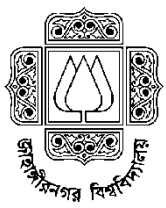
Concurrency Control in DDBMS

❑ Two-Phase Locking (2PL) Protocol

The 2PL protocol states that no transaction should acquire a lock after it releases one of its lock. According to this protocol, the life time of each transaction is divided into two phases:

- Growing phase and
- Shrinking phase.

In growing phase, a transaction can obtain locks on data items and can access data items, but it can not release any locks.



Concurrency Control in DDBMS

❑ Two-Phase Locking (2PL) Protocol

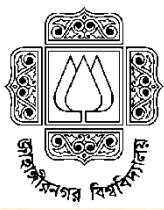
In shrinking phase, a transaction can release locks but cannot acquire any new locks after that. Thus, the ending of growing phase of a transaction determines the beginning of the shrinking phase of that transaction. It is not necessary for each transaction to acquire all locks simultaneously and then start processing. Normally, each transaction obtains some locks initially, does some processing and then requests for new additional locks that are required. However, it never releases any lock until it has reached a stage where no more locks are required. If up-gradation and down-gradation are allowed, then up-gradation of locks can take place in the growing phase, whereas down-gradation of locks can occur in the shrinking phase.



Distributed Deadlock Management

❑ Deadlock

In a database environment, a deadlock is a situation when transactions are endlessly waiting for one another. Any lock-based concurrency control algorithm and some timestamp-based concurrency control algorithms may result in deadlocks, as these algorithms require transactions to wait for one another.



Distributed Deadlock Management

❑ Deadlock

Deadlock situations can be characterized by **wait-for graphs**, directed graphs that indicate which transactions are waiting for which other transactions.

In a wait-for graph, **nodes** of the graph represent transactions and edges of the graph represent the waiting-for relationships among transactions. An **edge** is drawn in the wait-for graph from transaction T_i to transaction T_j , if the transaction T_i is waiting for a lock on a data item that is currently held by the transaction T_j .

Using wait-for graphs, it is very easy to detect whether a deadlock situation has occurred in a database environment or not. **There is a deadlock in the system if and only if the corresponding wait-for graph contains a cycle.**



Distributed Deadlock Management

❑ Deadlock

- The resolution of a deadlock situation is much easier in a centralized DBMS than in a distributed DBMS. In a centralized DBMS, only one **local wait-for graph** (LWFG) is drawn to detect the deadlock situation. The detection of deadlocks in a distributed DBMS is more complicated, because the circular waiting situation, which determines a deadlock, may involve several different sites. Thus, in a distributed DBMS it is not sufficient to draw a LWFG for each local DBMS only, but it is also necessary to draw a **global wait-for graph** (GWFG) for the entire system to detect a deadlock situation. In a distributed database, an LWFG is a portion of the GWFG.



Distributed Deadlock Management

❑ Deadlock

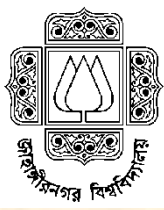
- Three general techniques are available for deadlock resolution in a distributed database system:
 - **Distributed deadlock prevention,**
 - **Distributed deadlock avoidance and**
 - **Distributed deadlock detection and recovery from deadlock.**



Distributed Deadlock Management

❑ Distributed Deadlock Prevention Method

Distributed Deadlock prevention is a cautious scheme in which a transaction is restarted when the system suspects that a deadlock might occur. Deadlock prevention is an alternative method to resolve deadlock situations in which a system is designed in such a way that **deadlocks are impossible**. In this scheme, the **transaction manager** checks a transaction when it is first initiated and does not permit to proceed if there is a risk that it may cause a deadlock.



Distributed Deadlock Management

❑ Distributed Deadlock Prevention Method

In the case of **lock-based concurrency control**, deadlock prevention in a distributed system is implemented in the following way:

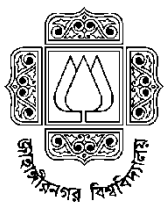
Let us consider that a transaction T_i is initiated at a particular site in a distributed database system and that it requires a lock on a data item that is currently owned by another transaction T_j . Here, a deadlock prevention test is done to check whether there is any possibility of a deadlock occurring in the system. The transaction T_i is not permitted to enter into a wait state for the transaction T_j , if there is the risk of a deadlock situation. In this case, one of the two transactions is aborted to prevent a deadlock.



Distributed Deadlock Management

❑ Distributed Deadlock Prevention Method

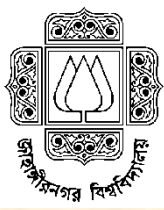
The deadlock prevention algorithm is called **non-preemptive** if the transaction T_i is aborted and restarted. On the other hand, if the transaction T_j is aborted and restarted, then the deadlock prevention algorithm is called **preemptive**. The transaction T_i is permitted to wait for the transaction T_j as usual, if they pass the prevention test. The prevention test must guarantee that if T_i is allowed to wait for T_j , a deadlock can never occur.



Distributed Deadlock Management

❑ Distributed Deadlock Prevention Method

A better approach to implement deadlock prevention test is by assigning priorities to transactions and checking priorities to determine whether one transaction would wait for the other transaction or not. These priorities can be assigned by using a unique identifier for each transaction in a distributed system. For instance, consider that i and j are priorities of two transactions T_i and T_j respectively. The transaction T_i would wait for the transaction T_j , if T_i has a lower priority than T_j , that is, if $i < j$. This approach prevents deadlock, but one problem with this approach is that cyclic restart is possible. Thus, some transactions could be restarted repeatedly without ever finishing.



Distributed Deadlock Management

❑ Distributed Deadlock Prevention Method

One solution to the above problem is using the unique timestamp value of each transaction as the priority of that transaction. One way to obtain a global timestamp value for every transaction in a distributed system is a unique local timestamp value to each transaction by its local transactions manager and then appending the site identifier to the low-order bites of this value. Thus, timestamp values are unique throughout the distributed system and do not require that clocks at different sites are synchronized precisely. **Based on timestamp values**, there are two different techniques for deadlock prevention: **Wait-die** and **Wound-wait**.



Distributed Deadlock Management

❑ Distributed Deadlock Prevention Method

Wait-die is a **non-preemptive** deadlock prevention technique based on timestamp values of transactions:

In this technique, when one transaction is about to block and is waiting for a lock on a data item that is already locked by another transaction, timestamp values of both the transactions are checked to **give priority to the older transaction**. If a younger transaction is holding the lock on data item then the older transaction is allowed to wait, but if an older transaction is holding the lock, the younger transaction is aborted and restarted with the same timestamp value. This forces the wait-for graph to be directed from the older to the younger transactions, making cyclic restarts impossible. For example, if the transaction T_i requests a lock on a data item that is already locked by the transaction T_j , then T_i is permitted to wait only if T_i has a lower timestamp value than T_j . On the other hand, if T_i is younger than T_j , then T_i is aborted and restarted with the same timestamp value.



Distributed Deadlock Management

❑ Distributed Deadlock Prevention Method

Wound-Wait is an alternative **preemptive** deadlock prevention technique by which cyclic restarts can be avoided.

In this method, if a younger transaction requests for a lock on a data item that is already held by an older transaction, the younger transaction is allowed to wait until the older transaction releases the corresponding lock. In this case, the wait-for graph flows from the younger to the older transactions, and cyclic restart is again avoided. For instance, if the transaction T_i requests a lock on a data item that is already locked by the transaction T_j , then T_i is permitted to wait only if T_i has a higher timestamp value than T_j , otherwise, the transaction T_j is aborted and the lock is granted to the transaction T_i .



Distributed Deadlock Management

❑ Distributed deadlock detection and recovery Method

Deadlock detection and recovery is the most popular and most suitable technique for deadlock management in a database environment.

In deadlock detection and recovery method, first it is checked whether any deadlock has occurred in the system. After detection of a deadlock situation in the system, one of the involved transactions is chosen as the victim transaction and is aborted to resolve the deadlock situation. Deadlock situations are detected by explicitly constructing a wait-for graph and searching it for cycles. A cycle in the wait-for graph indicates that a deadlock has occurred, and one transaction in the cycle is chosen as the victim, which is aborted and restarted. To minimize the cost of restarting, the victim selection is usually based on the number of data items used by each transaction in the cycle.



Distributed Deadlock Management

❑ Distributed deadlock detection and recovery Method

The principal difficulty in implementing deadlock detection in a distributed database environment is constructing the GWFG efficiently. In a distributed DBMS, a LWFG for each local DBMS can be drawn easily; however, these LWFGs are not sufficient to represent all deadlock situations in the distributed system. For example, consider that there are three different sites in a distributed system, and each site has constructed a LWFG as shown in figure. The LWFG for each site is constructed in the usual manner using local transactions and data items stored at that particular site. A cycle in a LWFG indicates that a deadlock has occurred locally. The LWFGs in Fig. illustrate that no deadlock has occurred locally in the three different sites, as there are no cycles in the LWFGs, but this does not guarantee that no deadlock has occurred globally. To detect a deadlock situation in the distributed system, it is necessary to construct a GWFG from these different LWFGs and to search it for cycles.



Distributed Deadlock Management

❑ Distributed deadlock detection and recovery Method

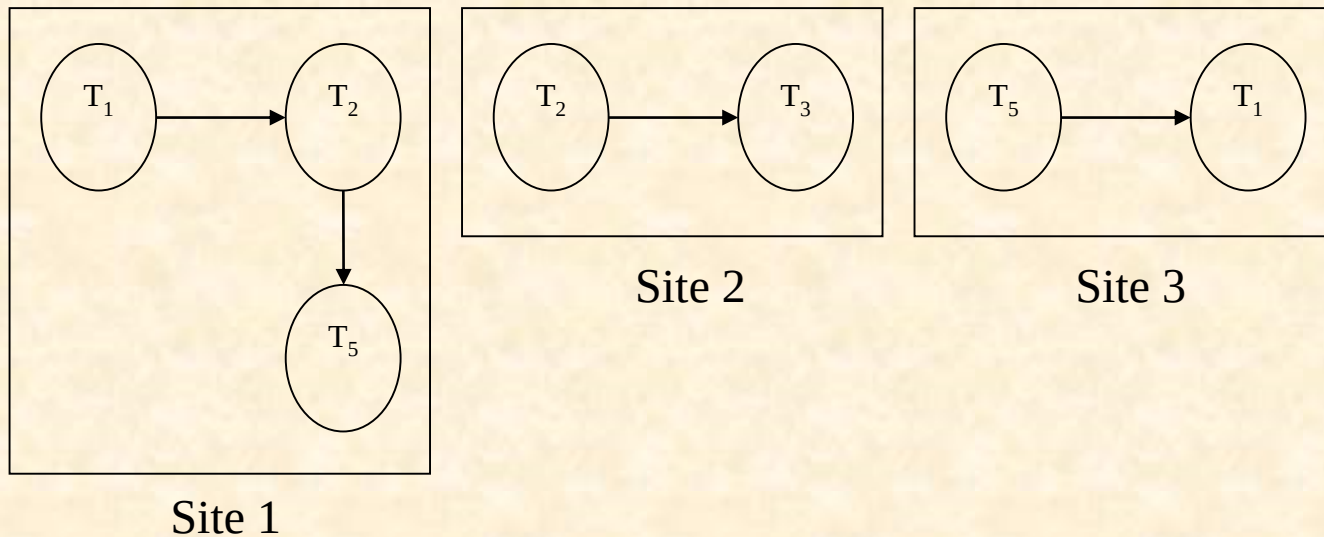


Fig. : Local Wait-For Graphs (LWFGs) at different Sites



Distributed Deadlock Management

❑ Distributed deadlock detection and recovery Method

The corresponding GWFG in Fig. illustrates that a deadlock has occurred in the distributed system, although no deadlock has occurred locally.

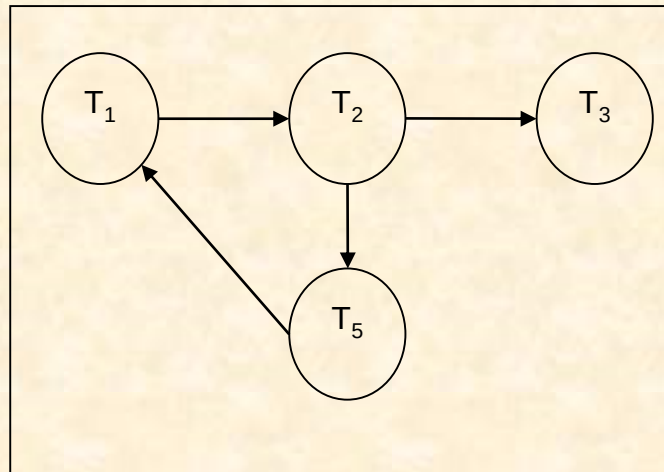


Fig.: Global Wait-For Graph (GWFG) for the above Fig.

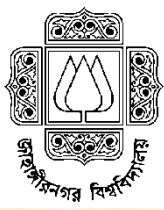


Distributed Deadlock Management

❑ Distributed deadlock detection and recovery Method

There are three different techniques for detecting deadlock situations in a distributed system:

- **Centralized deadlock detection,**
- **Hierarchical deadlock detection and**
- **Distributed deadlock detection**



Distributed Deadlock Management

❑ Centralized Deadlock detection

In Centralized Deadlock detection method, a single site is chosen as **Deadlock Detection Coordinator (DDC)** for the entire distributed system. The DDC is responsible for constructing the GWFG for system. Each **lock manager** in the distributed database transmits its LWFG to the DDC periodically. The DDC constructs the GWFG from these LWFGs and checks for cycles in it. The occurrence of a global deadlock situation is detected if there are one or more cycles in the GWFG. The DDC must break each cycle in the GWFG by selecting the transactions to be rolled back and restarted to recover from a deadlock situation. The information regarding the transactions that are to be rolled back and restarted must be transmitted to the corresponding lock managers by the deadlock detection coordinator.



Distributed Deadlock Management

❑ Centralized Deadlock detection

- The centralized deadlock detection approach is very simple, but it has several drawbacks.
- This method is **less reliable**, as the failure of the central site makes the deadlock detection impossible.
- The **communication cost is very high** in the case, as other sites in the distributed system send their LWFGs to the central site.
- Another disadvantage of centralized deadlock detection technique is that **false detection of deadlocks** can occur, for which the deadlock recovery procedure may be initiated, although no deadlock has occurred. In this method, unnecessary rollbacks and restarts of transactions may also result owing to phantom deadlocks.



Distributed Deadlock Management

❑ Distributed Deadlock detection

In distributed deadlock detection method, a deadlock detector exists at each site of the distributed system. In this method, each site has the same amount of responsibility, and there is no such distinction as local or global deadlock detector. A variety of approaches have been proposed for distributed deadlock detection algorithms, but the most well-known and simplified version, which is presented here, was developed by R. Obermarck in 1982. The centralized deadlock detection approach is very simple,



Distributed Deadlock Management

❑ Distributed Deadlock detection

In this approach, a LWFG is constructed for each site by the respective local deadlock detectors. An additional external node is added to the LWFGs, as each site in the distributed system receives the potential deadlock cycles from other sites. In the distributed deadlock detection algorithm, the external node T_{ex} is added to the LWFGs to indicate whether any transaction from any remote site is waiting for a data item that is being held by a transaction at the local site or whether any transaction from the local site is waiting for a data item is currently being used by any transaction at any remote site.



Distributed Deadlock Management

□ Distributed Deadlock detection

For instance, an edge from the node T_i to T_{ex} exists in the LWFG, if the transaction T_i is waiting for a data item that is already held by any transaction at any remote site. Similarly, an edge from the external node T_{ex} to T_i exists in the graph, if a transaction from a remote site is waiting to acquire a data item that is currently being held by the transaction T_i at the local site.



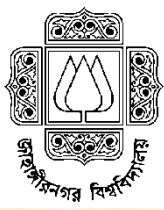
Distributed Deadlock Management

❑ Distributed Deadlock detection

Thus, the local detector checks for two things to determine a deadlock situation.

If a LWFG contains a cycle that does not involve the external node T_{ex} , then it indicates that a deadlock has occurred locally and it can be handled locally.

On the other hand, a global deadlock potentially exists if the LWFG contains a cycle involving the external node T_{ex} . However, the existence of such a cycle does not necessarily imply that there is a global deadlock, as the external node T_{ex} represents different agents.

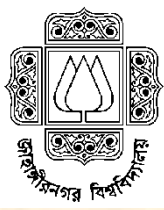


Distributed Deadlock Management

❑ Distributed Deadlock detection

The LWFGs are merged so as to determine global deadlock situations. To avoid sites transmitting their LWFGs to each other, a simple strategy is followed here.

According to this strategy, one timestamp value is allocated to each transaction and a rule is imposed such that one site S_i transmits its LWFG to the site S_k , if a transaction, say T_k , at site S_k is waiting for a data item that is currently being held by a transaction T_i at site S_i and $ts(T_i) < ts(T_k)$. If $ts(T_i) < ts(T_k)$, the site S_i transmits its LWFG to the site S_k , and the site S_k adds this information to its LWFG and checks for cycles not involving the external the node T_{ex} in the extended graph. If there is no cycle in the extended graph, the process continues until a cycle appears and it may happen that the entire GWFG is constructed and no cycle is detected. In this case, it is decided that there is no deadlock in the entire distributed system.



Distributed Deadlock Management

❑ Distributed Deadlock detection

On the other hand, if the GWFG contains a cycle not involving the external node T_{ex} , it is concluded that a deadlock has occurred in the system. The distributed deadlock detection method is illustrated below.

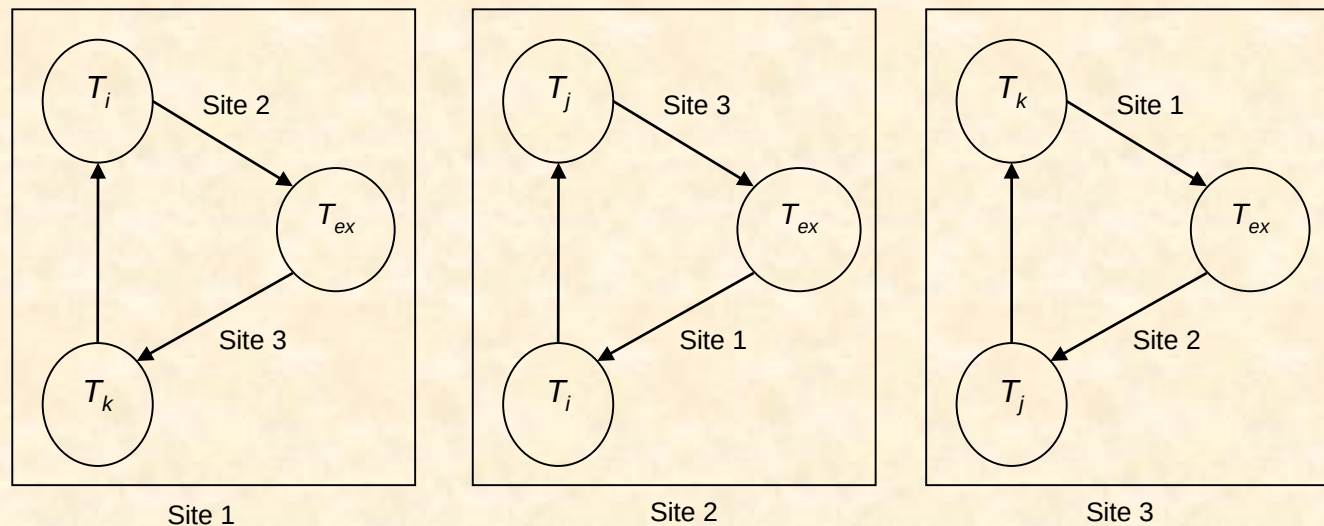


Fig. Distributed Deadlock Detection



Distributed Deadlock Management

❑ False Deadlocks

To handle the deadlock situation in distributed database systems, a number of messages are transmitted among the sites. The delay associated with the transmission of messages that is necessary for deadlock detection can cause the detection of **False Deadlocks**.

For instance, consider that at a particular time the deadlock detector has received the information that the transaction T_i is waiting for the transaction T_j . Further assume that after some time the transaction T_j releases the data item requested by the transaction T_i and requests for data item that is being currently held by the transaction T_i . If the deadlock detector receives the information that the transaction T_j has requested for a data item that is held by the transaction T_i before receiving the information that the transaction T_i is not blocked by the transaction T_j any more, a false deadlock situation is detected.



Distributed Deadlock Management

❑ Phantom Deadlocks

Another problem is that a transaction T_i that blocks another transaction may be restarted for reasons that are not related to deadlock detection. In this case, until the restart message of the transaction T_i is transmitted to the deadlock detector, the deadlock detector can find a cycle in the wait-for graph that includes the transaction T_i . Hence, a deadlock situation is detected by the deadlock detector and this is called a **Phantom Deadlock**. When the deadlock detector detects a phantom deadlock, it may unnecessarily restart a transaction other than T_i . To avoid unnecessary restarts for phantom deadlock, special safety measures are required.